CAHIER DES CHARGES TECHNIQUE

Microservice Finances

Double flux entrant/sortant, abonnement plateforme, hibernation sur impayé — séparation des tâches et journalisation immuable

Mirco Houénangon S.

1. Présentation du microservice

1.1. Contexte et rôle dans la plateforme

Le microservice Finances est le module le plus sensible de la plateforme : il porte, dans une seule table finances.transactions, l'ensemble des flux d'argent — ce qu'un établissement encaisse (frais de scolarité, cantine, transport, frais de dossier) et ce qu'il décaisse (remboursements, paiements fournisseurs, paie du personnel) — ainsi que le paiement de l'abonnement de l'établissement à la plateforme elle-même. Cette table unique, distinguée par une colonne sens (ENTRANT/SORTANT), garantit une vue de trésorerie cohérente sans rapprochement manuel entre systèmes séparés.

Le compte rendu de réunion du 11 septembre a formalisé cette distinction en deux niveaux de flux : Parents → Établissement (frais de scolarité, cantine, activités) et Établissement → Plateforme (abonnement) — les deux étant portés par ce même microservice mais avec des règles de gouvernance distinctes (voir sections 4 et 5).

1.2. Objectifs du microservice

- centraliser tout mouvement financier d'un établissement dans une source de vérité unique, immuable et auditable ;
- encaisser les frais de scolarité et frais annexes via mobile money (FedaPay, KkiaPay) avec confirmation automatique par webhook, sans saisie manuelle ;
- encadrer tout décaissement par une règle de séparation des tâches (initiateur ≠ validateur), activée par défaut ;
- distinguer clairement les paiements de scolarité (niveau établissement) des paiements d'abonnement à la plateforme (niveau plateforme) ;
- déclencher automatiquement un mode de fonctionnement restreint (« hibernation ») en cas d'impayé de l'abonnement plateforme, sans pénaliser injustement les familles ;
- produire des rapports financiers fiables, par établissement et consolidés au niveau plateforme, sans agrégation dupliquée à maintenir séparément.

1.3. Périmètre du document

Ce document couvre l'intégralité des flux financiers (entrants et sortants), la facturation, les intégrations de paiement, le mécanisme d'hibernation sur impayé, l'audit financier et le reporting. Il ne couvre pas la définition des frais de scolarité en tant que politique tarifaire pédagogique (portée par l'administrateur de l'établissement via Scolarité), ni la comptabilité générale d'entreprise multi-devises complète (explicitement hors périmètre V1 du cahier des charges plateforme).

2. État de l'art et positionnement

2.1. Solutions existantes

- EcoleDirecte (France) — intègre un suivi des paiements de cantine et d'activités, mais via des moyens de paiement européens (carte bancaire, prélèvement SEPA) sans équivalent mobile money, et sans logique de décaissement (paie, remboursements fournisseurs) ;
- KiboERP — encaissement mobile money natif (Wave, Orange Money, MTN MoMo) pour la scolarité en zone UEMOA, fonctionnalité la plus proche du périmètre encaissement de ce microservice, mais sans documentation publique d'un flux de décaissement à double validation ni d'un mécanisme d'abonnement plateforme avec état d'hibernation ;
- Novacole, Academia Helm — solutions ouest-africaines couvrant les paiements de scolarité et, pour Academia Helm, la fiabilité financière (reçus numérotés, clôture de caisse) ; aucune ne documente publiquement une architecture à double flux dans une table unique, ni une séparation des tâches systématique sur les décaissements ;
- les solutions génériques de comptabilité (Sage, tableurs) sont utilisées en parallèle par de nombreux établissements pour la partie décaissement, faute d'outil intégré — créant une double saisie et un risque de désynchronisation que ce microservice cherche justement à éliminer.

2.2. Problème identifié

Le marché ouest-africain a largement résolu l'encaissement mobile money côté scolarité, mais aucune solution recensée ne documente publiquement une gestion unifiée des décaissements (remboursements, paie, fournisseurs) avec séparation des tâches obligatoire, ni un mécanisme gradué de restriction d'accès en cas d'impayé d'abonnement qui protège les familles pendant que l'administration de l'établissement est suspendue.

2.3. Valeur ajoutée du positionnement retenu

- une seule table de transactions pour l'intégralité des flux (entrants et sortants, scolarité et abonnement plateforme), éliminant tout rapprochement manuel entre systèmes séparés ;
- une séparation des tâches systématique et paramétrable sur tout décaissement, avec garde-fou explicite en cas de désactivation (section 6.1) ;
- un état d'hibernation intermédiaire — plutôt qu'une coupure brutale — qui protège l'accès des familles pendant qu'un établissement régularise son abonnement (section 4).

3. Principes fonctionnels retenus

3.1. Matrice acteur × action

Acteur	Actions possibles	Restriction principale
Parent	Payer les frais de scolarité et frais annexes, consulter l'historique de facturation	Ses enfants rattachés uniquement
Comptable	Initier un décaissement, rapprocher les paiements espèces, produire les rapports financiers	Ne peut jamais valider un décaissement qu'il a lui-même initié
Administrateur (Fondateur)	Valider un décaissement initié par le comptable, configurer les formules tarifaires, paramétrer la règle de séparation des tâches	Limité à son tenant_id
Super Administrateur	Consulter les revenus consolidés, gérer les plans/tarifs, déclencher/lever l'hibernation d'un établissement	Aucun accès aux transactions internes d'un établissement hors contexte de support

Tableau 1 — Matrice des droits par acteur

3.2. Règles générales, non négociables

- toute transaction, entrante ou sortante, est une ligne dans finances.transactions, jamais dans une table séparée par nature de flux ;
- une transaction, une fois confirmée, n'est jamais modifiée ni supprimée : toute correction est une nouvelle transaction liée à l'originale ;
- aucun décaissement ne peut être exécuté sans validation par un second profil habilité — règle paramétrable par le fondateur mais activée par défaut ;
- le paiement de l'abonnement plateforme et les paiements de scolarité utilisent les mêmes agrégateurs mais restent des transactions distinctes, séparées par leur portée ;
- tout encaissement mobile money est confirmé exclusivement par webhook de l'agrégateur, jamais par une saisie manuelle de confirmation.

4. Abonnement plateforme et hibernation sur impayé

DÉCISION DE RÉUNION (11/09/2026) — en cas d'expiration du forfait d'un établissement, le comité de pilotage a acté l'application d'un mode de fonctionnement minimal : accès restreint aux fonctionnalités vitales côté parents, et passage en hibernation complète côté administration de l'établissement. Cette section formalise cette décision, absente du cahier des charges plateforme initial (qui ne prévoyait qu'une suspension binaire actif/suspendu).

4.1. Principe : trois états au lieu de deux

Le cahier des charges plateforme initial ne prévoit que deux états pour un établissement : actif ou suspendu. La décision de réunion introduit un état intermédiaire, l'hibernation, déclenché spécifiquement par un impayé d'abonnement (par opposition à une suspension décidée manuellement pour un autre motif). Ce microservice Finances est le déclencheur de cet état, mais son application effective reste portée par le microservice Établissements, propriétaire du statut du tenant.

4.2. Ce que déclenche l'hibernation

Côté	Comportement en hibernation
Parents / Apprenants	Accès restreint mais maintenu aux fonctionnalités vitales : consultation des notes déjà publiées, des bulletins existants, paiement des frais en attente — mais pas de nouvelle inscription, pas de nouvelle demande administrative
Personnel / Administration de l'établissement	Hibernation complète : aucun accès au back-office, aucune saisie possible, jusqu'à régularisation

Tableau 2 — Comportement différencié selon le profil en mode hibernation

4.3. Cycle de vie de l'abonnement plateforme

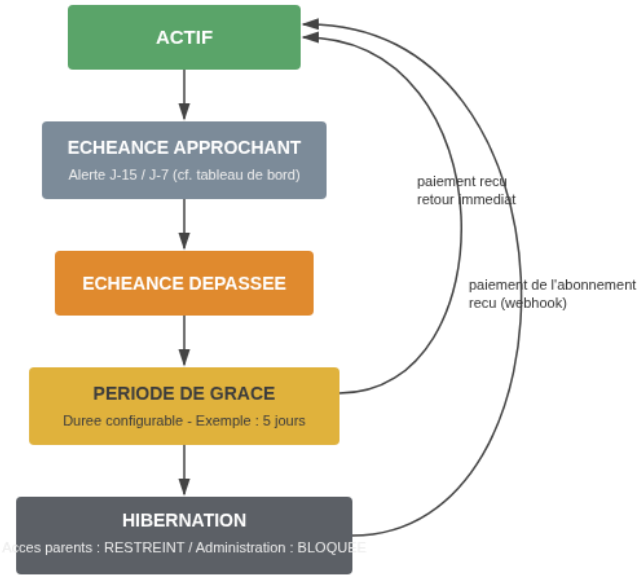


Figure 1 — Cycle de vie de l'abonnement plateforme d'un établissement

PROPOSITION (hors cahier des charges plateforme initial) — introduire une période de grâce configurable entre l'échéance dépassée et l'entrée effective en hibernation, pour absorber les délais bancaires ou un simple oubli de renouvellement — point non détaillé dans le compte rendu

de réunion, à valider avec le super administrateur (Don Diègue, Établissements) car la bascule d'état reste sous sa responsabilité.

5. Flux entrants (encaissements)

- définition des frais de scolarité par classe/formule (formules_tarifaires), avec échéanciers ;
- relances automatiques avant et après échéance, avec paliers configurables ;
- encaissement des frais de scolarité, de dossier/demande administrative, de cantine et de transport, dans la même table transactions ;
- intégration native de FedaPay et KkiaPay (MTN MoMo, Moov Money, cartes bancaires selon les moyens supportés par chaque agrégateur) ;
- paiement en espèces encadré : saisi et rapproché par le comptable ou le personnel habilité, avec pièce justificative ;
- génération de factures et reçus, avec accusé de paiement automatique dès confirmation de l'agrégateur (webhook).

6. Flux sortants (décaissements)

- remboursement d'un parent (trop-perçu, annulation, avoir) ;
- paiement d'un prestataire externe, en lien direct avec le module Relations prestataires ;
- versement de la paie du personnel, en lien avec le module Ressources Humaines ;
- tout autre paiement sortant initié par le comptable ou le fondateur, avec validation obligatoire avant exécution ;
- exécution par mobile money (via FedaPay/KkiaPay lorsque l'agrégateur le permet en sortant, ou par API opérateur directe) ou par virement bancaire classique ;
- statut de suivi du décaissement : initié → validé → exécuté → échoué, avec journalisation immuable.

6.1. Règle de contrôle (séparation des tâches)

Aucun décaissement ne peut être exécuté sans une étape de validation par un second profil habilité : la personne qui initie un paiement sortant n'est jamais, par défaut, celle qui le valide définitivement. Cette règle est paramétrable par le fondateur mais activée par défaut.

POINT DE VIGILANCE — GARDE-FOU À AJOUTER — le cahier des charges plateforme laisse la règle de séparation des tâches désactivable par le fondateur, sans garde-fou décrit. Il est recommandé d'ajouter, dès la V1, une alerte automatique au super administrateur (visible dans son

tableau de bord et journalisée) chaque fois qu'un établissement désactive cette règle, ainsi qu'un contrôle renforcé (ex. plafond de décaissement sans double validation, configurable).

PROPOSITION (hors cahier des charges plateforme initial) — le plafond de décaissement sans double validation n'est pas dans le cahier des charges initial ; il est proposé comme mesure de sécurité complémentaire, activable établissement par établissement.

7. Intégrations de paiement (FedaPay, KkiaPay, mobile money, virement, espèces)

Moyen de paiement	Sens	Mécanisme de confirmation
FedaPay (MTN MoMo, Moov Money, carte)	Entrant et sortant (selon capacité de l'agrégateur)	Webhook automatique, idempotent
KkiaPay (MTN MoMo, Moov Money, carte)	Entrant et sortant (selon capacité de l'agrégateur)	Webhook automatique, idempotent
Virement bancaire classique	Sortant uniquement	Confirmation manuelle par le comptable, avec pièce justificative
Espèces	Entrant uniquement	Saisie et rapprochement manuel par le comptable, encadrés
API opérateur directe (hors agrégateur)	Sortant, si l'agrégateur ne couvre pas ce cas	Statut de suivi manuel jusqu'à confirmation opérateur

Tableau 3 — Moyens de paiement et mécanismes de confirmation

7.1. Idempotence des webhooks

Chaque webhook reçu est traité de façon idempotente (identifiant de référence externe unique stocké et vérifié) pour éviter qu'une notification dupliquée par l'agrégateur ne crée deux fois la même transaction — un cas fréquent en environnement mobile money avec latence réseau variable.

8. Facturation, reçus et modèles de documents

Sur le même principe que les modèles de bulletins (Évaluations) et de documents administratifs (Demandes administratives), chaque établissement peut personnaliser la mise en page de ses factures et reçus (en-tête, mentions légales, cachet). Le moteur de rendu est, ici encore, un candidat à la mutualisation évoquée dans les deux autres cahiers des charges.

PROPOSITION (hors cahier des charges plateforme initial) — mutualiser un unique moteur de templates PDF, partagé entre Évaluations, Demandes administratives et Finances — les trois modules étant portés majoritairement par la même personne, cette mutualisation est simple à mettre en œuvre dès la conception.

9. Audit financier et rapprochement comptable

DÉCISION DE RÉUNION (11/09/2026) — le comité de pilotage a demandé un cadrage strict des types d'audits comptables et des mécanismes d'isolation dans le module Finances. Cette section répond directement à cette demande.

9.1. Types d'audits pris en charge

- audit de cohérence : toute transaction confirmée doit correspondre à une écriture webhook ou à une pièce justificative rapprochée (aucune transaction « orpheline ») ;
- audit de séparation des tâches : détection automatique de tout décaissement où l'initiateur et le validateur seraient identiques ;
- audit d'isolation multi-tenant : vérification que toute transaction porte un tenant_id cohérent et qu'aucune requête ne peut croiser deux établissements ;
- audit de rapprochement espèces : suivi des écarts entre les espèces déclarées encaissées et les pièces justificatives physiques associées.

9.2. Journalisation

Chaque transaction, chaque décision de validation/rejet de décaissement, et chaque export de rapport financier est journalisé dans l'audit_log centralisé de la plateforme, avec identité de l'auteur, horodatage, et contenu de l'action.

10. Rapports financiers et indicateurs

- rapports financiers par établissement (recettes, dépenses, solde), calculés directement sur finances.transactions, filtrés par sens et par période ;
- recettes de scolarité par classe, par jour et par moyen de paiement ;
- au niveau plateforme : abonnements, renouvellements, revenus par module, MRR/ARR équivalent, taux de churn des établissements ;
- aucune agrégation dupliquée à maintenir séparément — la table unique sert à la fois de journal comptable et de source de reporting.

11. Expérience utilisateur

- paiement de scolarité en un minimum d'étapes depuis la PWA : sélection de l'enfant, montant dû affiché clairement, choix du moyen de paiement, confirmation immédiate dès webhook reçu ;
- tableau de bord comptable avec vue de réconciliation claire ;
- double validation de décaissement présentée comme un flux guidé (l'écran du validateur affiche explicitement qui a initié, pourquoi, et le montant) ;
- alerte visuelle et notification pour tout établissement approchant l'échéance de son abonnement plateforme.

PROPOSITION (hors cahier des charges plateforme initial) — proposer, pour les parents, un échéancier de paiement fractionné configurable par établissement (ex. paiement en 3 fois) plutôt qu'un paiement en un seul versement.

12. Acteurs et cas d'utilisation

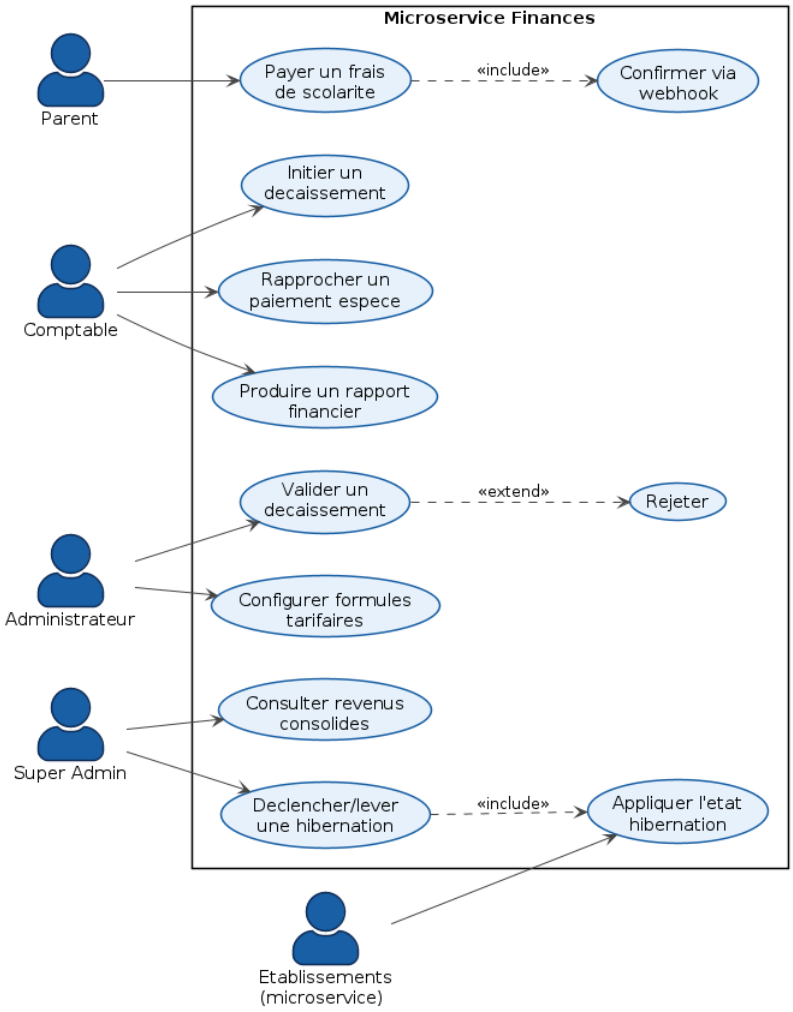


Figure 2 — Diagramme de cas d'utilisation du microservice Finances

Cas d'utilisation	Description	Post-condition
Payer un frais de scolarité	Le parent règle une facture via FedaPay/KkiaPay	Transaction ENTRANT créée, statut confirmé au webhook
Initier un décaissement	Le comptable crée une demande de paiement sortant	Transaction SORTANT créée, statut « initié »
Valider un décaissement	Un second profil habilité approuve la demande	Statut « validé », exécution déclenchée
Exécuter un décaissement	Paiement effectif par mobile money ou virement	Statut « exécuté » ou « échoué »
Produire un rapport financier	Le comptable ou l'administrateur exporte un rapport filtré	Aucune écriture, lecture agrégée
Déclencher une hibernation	Le système constate un impayé d'abonnement au-delà de la période de grâce	Événement transmis à Établissements pour bascule d'état

Tableau 4 — Détail des principaux cas d'utilisation

13. Modèle de données du schéma « finances »

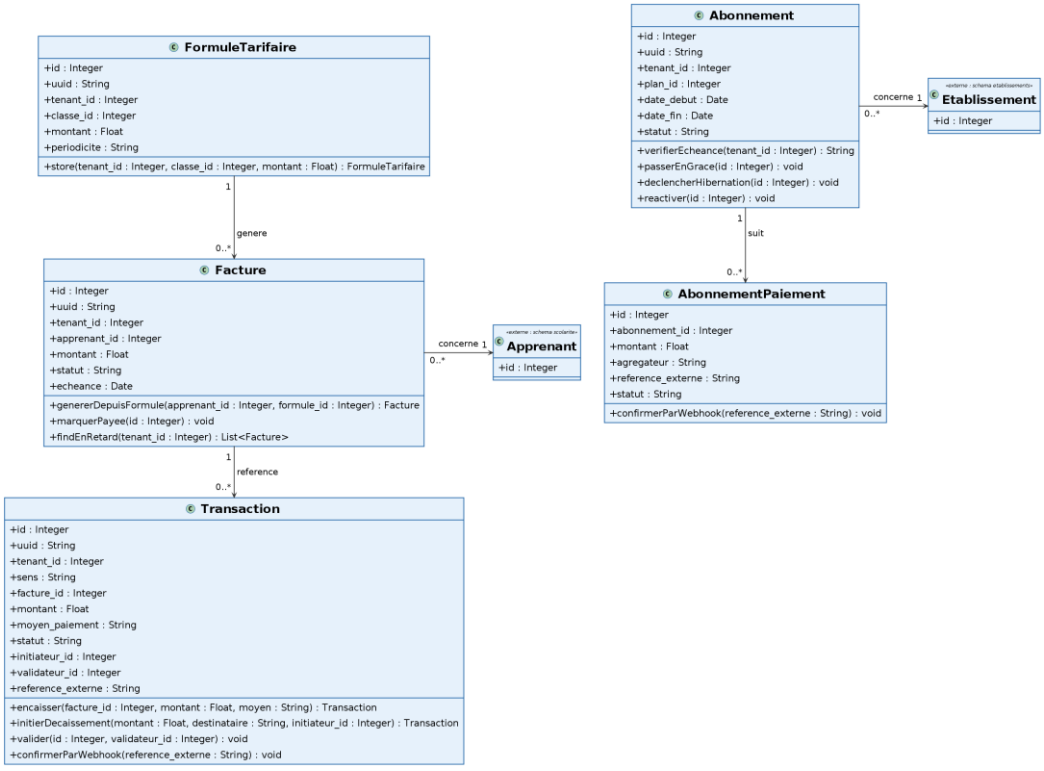


Figure 3 — Diagramme de classes simplifié du schéma finances

13.1. Détail des tables

Table	Rôle	Champs clés
formules_tarifaires	Frais de scolarité par classe/formule, niveau établissement	tenant_id, classe_id, montant, periodicite
factures	Facturation des frais dus par l'apprenant, avec échéance	tenant_id, apprenant_id, montant, statut, echeance
transactions	Table unique de tout flux financier entrant ou sortant, journalisée de façon immuable	tenant_id, sens, facture_id, montant, moyen_paiement, destinataire, statut, initiateur_id, validateur_id, reference_externe, date
abonnements	Abonnement en cours de l'établissement à la plateforme, avec statut incluant « grâce » et « hibernation »	tenant_id, plan_id, date_debut, date_fin, statut
abonnements_paiements	Règlements de l'abonnement plateforme, confirmés par webhook	abonnement_id, montant, agregateur, reference_externe, statut

Tableau 5 — Détail des tables du schéma finances

14. Architecture technique

```
Web App / PWA        Passerelle API (Gateway)
     |                        |
     +------------------------+
                |
     Microservice Finances - Laravel
     +-- Controleurs API (transactions, factures, abonnements)
     +-- Service Encaissement (webhooks FedaPay/KkiaPay, idempotence)
     +-- Service Decaissement (workflow initiation/validation/execution)
     +-- Service Abonnement (echeances, periode de grace, hibernation)
     +-- Service Rapports (agregation finances.transactions)
```

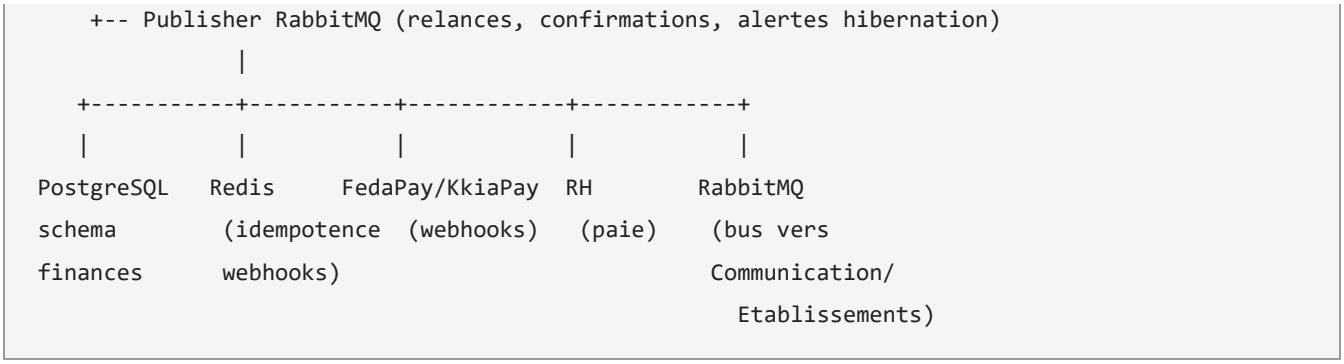


Figure 4 — Architecture interne du microservice Finances

- Service Encaissement : réceptionne et valide les webhooks, garantit l'idempotence, crée la transaction confirmée et met à jour la facture liée dans la même opération ;
- Service Décaissement : orchestre le cycle initié → validé → exécuté, applique la règle de séparation des tâches ;
- Service Abonnement : suit les échéances, gère la période de grâce, et notifie Établissements lorsqu'un impayé doit déclencher l'hibernation ;
- Service Rapports : agrège finances.transactions à la demande, sans table de reporting séparée.

14.1. RabbitMQ — bus d'événements asynchrones

RETOUR DU COMITÉ DE PILOTAGE (relecture des CDC) — RabbitMQ remplace le mécanisme d'événement applicatif interne pour toute notification asynchrone. Les jointures SQL directes utilisées pour les vérifications temps réel restent inchangées, s'exécutant sur la même base sans appel réseau.

- Finances publie sur des échanges RabbitMQ dédiés (ex. finances.paiement.confirme, finances.relance.due, finances.abonnement.hibernation) plutôt que d'appeler directement Communication ou Établissements ;
- le message finances.abonnement.hibernation est consommé par Établissements pour appliquer effectivement le changement de statut du tenant — Finances détecte l'impayé, Établissements applique le blocage ;
- ce découplage garantit qu'une indisponibilité momentanée de Communication ou d'Établissements ne fait perdre aucun événement — propriété particulièrement importante pour un événement aussi sensible qu'une hibernation.

15. Endpoints de l'API

Méthode	Endpoint	Description
POST	/factures	Crée une facture pour un apprenant selon une formule tarifaire
POST	/paiements/initier	Initie un encaissement via FedaPay/KkiaPay pour une facture
POST	/webhooks/fedapay	Réception du webhook FedaPay
POST	/webhooks/kkiapay	Réception du webhook KkiaPay
POST	/decaissements	Initie une demande de paiement sortant (comptable)
PUT	/decaissements/{uuid}/valider	Valide un décaissement initié par un autre profil
PUT	/decaissements/{uuid}/rejeter	Rejette une demande de décaissement, avec motif
GET	/rapports/etablissement/{tenant_uuid}	Rapport financier agrégé, filtrable par période et moyen de paiement
GET	/interne/solde/{apprenant_uuid}	Endpoint interne (Demandes administratives) : statut de solde d'un apprenant
GET	/abonnements/{tenant_uuid}	Statut d'abonnement (actif / grâce / hibernation) et date d'échéance
POST	/abonnements/{tenant_uuid}/paiement	Initie le paiement de l'abonnement plateforme
GET	/interne/revenus-consolides	Endpoint interne (super admin) : revenus consolidés par module et établissement

Tableau 6 — Endpoints exposés par le microservice Finances

16. Sécurité et conformité

- chiffrement systématique en transit et au repos pour toute donnée financière ;
- séparation stricte des tâches sur tout décaissement, avec alerte au super administrateur si la règle est désactivée (section 6.1) ;
- vérification de signature/authenticité de chaque webhook reçu (secret partagé avec l'agrégateur) ;
- idempotence stricte des webhooks pour éviter toute double comptabilisation ;

- aucune donnée de carte bancaire ne transite ni n'est stockée par la plateforme : la tokenisation est déléguée à FedaPay/KkiaPay ;
- aucun identifiant technique interne exposé en URL ;
- journalisation immuable de toute transaction et de toute décision de validation/rejet, consultable filtrée par tenant.

17. Relation avec les autres microservices

Finances est, avec Identité, l'un des deux microservices dont dépendent le plus grand nombre d'autres modules. C'est également le seul microservice, avec Évaluations, à exposer un endpoint consommé en retour par Demandes administratives.

```
        Etablissements --(jointure + evenement RabbitMQ)<--> Finances   (abonnement, hibernation)
        RH             --(jointure SQL directe, lecture)--> Finances   (versement paie)
        Relations prestataires --(jointure SQL directe)--> Finances   (paiement fournisseur)
        Scolarite      --(jointure SQL directe, lecture)--> Finances   (formules tarifaires par
classe)
        Identite       --(JWT via Gateway)-----------------> Finances
        Finances --(endpoint interne, lecture)<-- Demandes administratives   (verification solde)
        Finances --(RabbitMQ, evenement)---------> Communication   (relances, confirmations)
        Vie scolaire   --(jointure SQL directe, lecture)--> Finances   (paiement
cantine/transport)
```

Figure 5 — Relations entre le microservice Finances et les autres microservices

17.1. Établissements (Don Diègue) — relation bidirectionnelle critique

Type de communication : **jointure SQL directe en lecture** (Finances lit **etablissements.abonnements**) combinée à un **événement RabbitMQ** (finances.abonnement.hibernation) publié par Finances lorsqu'un impayé dépasse la période de grâce. C'est la relation la plus critique de ce module : Finances **détecte** l'impayé, mais c'est Établissements qui **applique** le changement de statut du tenant, puisqu'il est le propriétaire exclusif de la colonne statut. Cette séparation doit être précisément contractualisée avec Don Diègue avant développement.

17.2. Ressources Humaines (Arsène) — dépendance amont

Type de communication : **jointure SQL directe en lecture** sur **rh.personnel** pour le versement de la paie (décaissement sortant).

17.3. Relations prestataires (Oswald) — dépendance amont

Type de communication : **jointure SQL directe en lecture** sur **prestataires.contrats_prestataires** pour déclencher un paiement fournisseur, avec le même mécanisme de séparation des tâches.

17.4. Scolarité (Harold) — dépendance amont

Type de communication : **jointure SQL directe en lecture** sur **scolarite.classes** pour résoudre les formules tarifaires applicables à chaque classe.

17.5. Identité (Don Diègue) — authentification

Type de communication : **jeton JWT transmis par la Gateway**, porteur du rôle utilisé pour appliquer la règle de séparation des tâches.

17.6. Demandes administratives (Mirco) — relation entrante

Type de communication : **endpoint interne exposé par Finances** (**/interne/solde**), consommé par Demandes administratives pour les demandeurs de catégorie apprenant.

17.7. Communication (Don Diègue) — dépendance aval, via RabbitMQ

Type de communication : **événement publié sur RabbitMQ**, pour les relances, les confirmations de paiement et les alertes d'approche d'hibernation.

17.8. Vie scolaire (Paul) — dépendance amont légère

Type de communication : **jointure SQL directe en lecture** pour les formules de cantine et transport.

18. Exigences non fonctionnelles spécifiques

Exigence	Cible / seuil indicatif
Disponibilité	≥ 99,5 % — service financier critique, aligné sur l'exigence plateforme
Temps de confirmation d'un encaissement	< 3 secondes après réception du webhook
Idempotence des webhooks	100 % — aucune double comptabilisation possible
Auditabilité	100 % des transactions et décisions de validation journalisées, horodatées, non modifiables
Intégrité financière	Contraintes d'intégrité référentielle strictes au niveau base de données
Scalabilité	Absorption des pics de paiement en début de trimestre/rentrée
Fiabilité de l'événement d'hibernation	Aucune perte du message finances.abonnement.hibernation, même en cas d'indisponibilité momentanée d'Établissements (garantie de livraison RabbitMQ)

Exigence	Cible / seuil indicatif
Cohérence de l'hibernation	Propagation de l'état à l'ensemble des utilisateurs du tenant dès la requête suivante

Tableau 7 — Exigences non fonctionnelles du microservice Finances

19. Structure du projet et dockerisation

```
api-finances/
+-- app/
|   +-- Http/Controllers/  Transactions, Decaissements, Abonnements
|   +-- Services/
|   |   +-- EncaissementService.php   webhooks, idempotence
|   |   +-- DecaissementService.php   workflow initiation/validation
|   |   +-- AbonnementService.php     echeances, grace, hibernation
|   |   +-- RapportService.php
|   +-- Jobs/ RelanceAutomatique.php, VerifierEcheancesAbonnement.php
|   +-- Integrations/
|       +-- FedaPayClient.php, KkiaPayClient.php
|       +-- RhClient.php, PrestatairesClient.php, ScolariteClient.php
|       +-- RabbitMqPublisher.php   [NOUVEAU]
+-- database/migrations/ + seeders/
+-- tests/Unit/ (idempotence webhooks, separation des taches — priorite absolue)
+-- docker/Dockerfile
+-- docker-compose.yml
+-- .github/workflows/ci.yml
+-- .env.example, composer.json, README.md
```

Arborescence normalisée du dépôt api-finances

19.1. Dockerisation

RETOUR DU COMITÉ DE PILOTAGE (relecture des CDC) — le comité de pilotage a demandé que chaque microservice soit dockerisé.

- image de base : php:8.3-fpm-alpine, avec extensions pdo_pgsql, redis et amqp (client RabbitMQ) ;
- Dockerfile en deux étapes (build Composer puis image d'exécution allégée) ;
- docker-compose.yml local orchestre api-finances (port interne uniquement), avec dépendance vers PostgreSQL, Redis et RabbitMQ mutualisés au niveau plateforme ;

- variables d'environnement sensibles (clés FedaPay/KkiaPay live, JWT_PUBLIC_KEY, RABBITMQ_URL) injectées via GitHub Actions Secrets, jamais commitées, avec rotation périodique recommandée ;
- healthcheck HTTP (/health), incluant une vérification de connectivité à RabbitMQ et à l'agrégateur de paiement ;
- image publiée et taguée (sha du commit) à chaque fusion sur main, avec scan de vulnérabilités renforcé pour ce microservice.

20. Feuille de route et planning

Compte tenu de la sensibilité du module, la livraison est visée sur douze jours ouvrés, avec une équipe « Encaissements » et une équipe « Décaissements & abonnement » — cette dernière nécessitant une coordination étroite avec Don Diègue (Établissements).

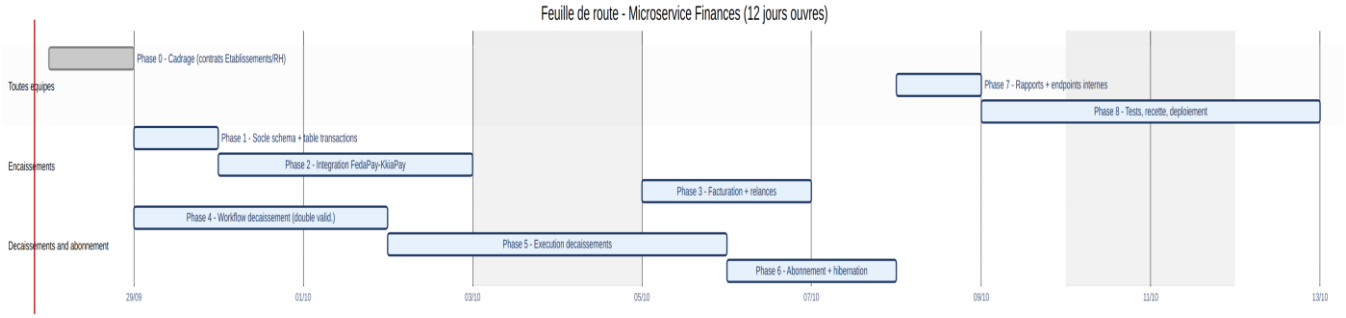


Figure 6 — Diagramme de Gantt du microservice Finances

Phase	Contenu	Durée	Équipe
0	Cadrage : contrat d'interface avec Établissements (hibernation, RabbitMQ) et RH/Prestataires	1 j	Toutes
1	Socle : schéma finances, migrations, table transactions unique	1 j	Encaissements
2	Intégration FedaPay/KkiaPay : encaissement, webhooks, idempotence	3 j	Encaissements
3	Facturation et échéanciers, relances automatiques (RabbitMQ)	1,5 j	Encaissements
4	Workflow de décaissement : initiation, séparation des tâches, validation	2,5 j	Décaissements & abonnement
5	Exécution des décaissements (mobile money sortant, virement)	1,5 j	Décaissements & abonnement

Phase	Contenu	Durée	Équipe
6	Abonnement plateforme : échéances, grâce, hibernation (RabbitMQ)	1,5 j	Décaissements & abonnement
7	Rapports financiers et endpoints internes	1 j	Toutes
8	Tests (priorité idempotence et séparation des tâches), recette, déploiement (image Docker)	2 j	Toutes

Tableau 8 — Détail des phases (environ 12 jours ouvrés)

21. Livrables attendus pour ce microservice

- dépôt Git dédié (api-finances), pipeline CI/CD GitHub Actions, image Docker publiée à chaque fusion sur main ;
- schéma de base de données finances détaillé (migrations, contraintes, index) ;
- intégrations FedaPay et KkiaPay opérationnelles, entrantes et sortantes, avec gestion idempotente des webhooks ;
- workflow de décaissement à double validation, testé avec couverture exhaustive des cas de rejet/erreur ;
- mécanisme d'abonnement plateforme avec période de grâce et déclenchement d'hibernation via RabbitMQ, contractualisé avec Établissements ;
- API REST documentée (OpenAPI/Swagger) ;
- tableau de bord comptable et tableau de bord de suivi d'abonnement ;
- moteur de rapports financiers par établissement et consolidés plateforme ;
- documentation technique du contrat d'interface avec Établissements, RH et Relations prestataires.

22. Conclusion

Le microservice Finances est le module le plus exposé de la plateforme : il manipule de l'argent réel, engage la confiance des familles et des établissements, et sa moindre défaillance a un impact direct et visible. La table unique de transactions, la séparation des tâches sur tout décaissement et l'introduction d'un état d'hibernation intermédiaire — propagé de façon fiable via RabbitMQ — en font un module à la fois rigoureux dans ses contrôles et respectueux des familles en cas d'impayé. Sa réussite dépend de deux coordinations externes précisément contractualisées : avec Établissements pour l'application de l'hibernation, et avec Demandes administratives pour la vérification de solde.